

React Native Mobile App Development

5-Day Beginner React Native Course

Table of Contents (Facilitator)

1. [Facilitator Preparation](#)
2. [Day 1: React Native Foundations - Facilitator Guide](#)
3. [Day 2: Core Components, Styling & Layout - Facilitator Guide](#)
4. [Day 3: Lists, Forms & User Interactions - Facilitator Guide](#)
5. [Day 4: Navigation & State Management - Facilitator Guide](#)
6. [Day 5: Native Features, Debugging & Publishing - Facilitator Guide](#)
7. [Appendix: Answer Keys](#)
8. [Appendix: Environment Setup Guide](#)

Facilitator Preparation

Master Props Checklist

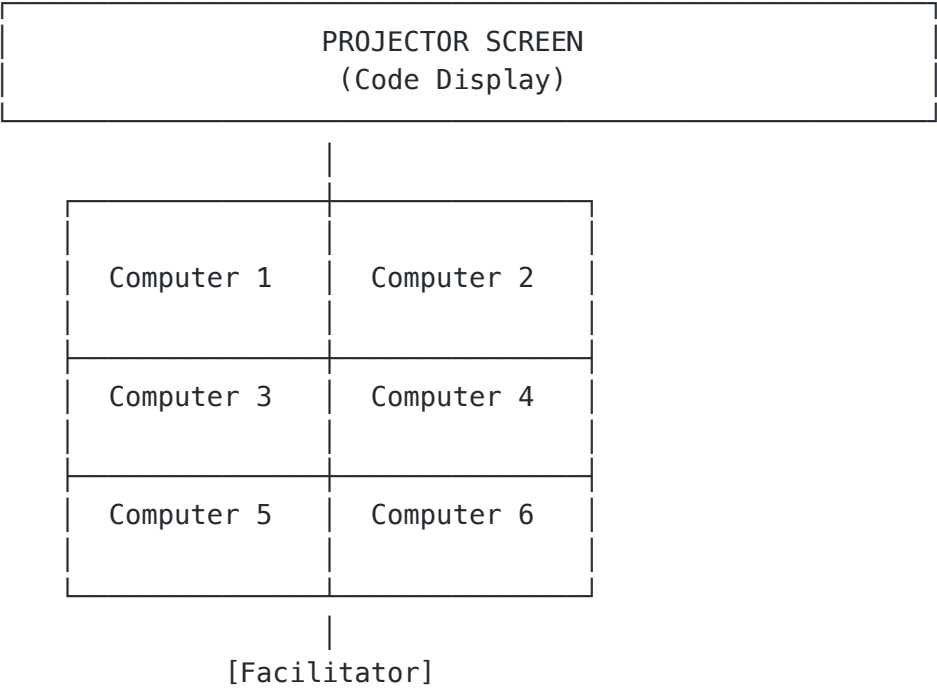
Item	Quantity	Purpose	Day Used
Computers with Node.js installed	1 per participant	Running React Native	All
VS Code with React Native extensions	1 per computer	Code editing	All
Projector/Display	1	Code demonstrations	All
Whiteboard + markers	1 large	Diagrams and concepts	All
Printed worksheets	1 set per participant	Exercises	All
Timer	1	Activity timing	All
React Native cheat sheets	1 per participant	Reference	All
Android phones (optional)	1-2	Physical device testing	Day 5

Installation Checklist

Before Day 1, ensure:

- ☐ Node.js 18+ installed on all computers
- ☐ Expo CLI installed: `npm install -g expo-cli` (or use `npx`)
- ☐ Android Studio with emulator (for Android testing)
- ☐ Xcode with iOS simulator (for Mac users)
- ☐ VS Code with React Native Tools extension
- ☐ Expo Go app installed on phones (optional)
- ☐ Test that `npx expo start` works
- ☐ Test that `npx expo run:android` works (if using Android)
- ☐ Internet access for npm packages

Classroom Setup



Day 1: Facilitator Guide

React Native Foundations & Environment Setup

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Opening & Environment Setup Check	15 min
09:15-09:45	React Native Overview & Architecture	30 min
09:45-10:15	EXERCISE: Setup Check	30 min
10:15-10:45	Core Components & JSX	30 min
10:45-11:15	EXERCISE: Hello World App	30 min
11:15-11:45	Project Structure & Expo	30 min
11:45-12:15	EXERCISE: Explore Project	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Building First App	60 min
14:00-15:00	BUILD: Component Practice	60 min
15:00-15:30	Day 1 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Opening & Setup Check

[SAY] "Good morning everyone. Welcome to React Native Mobile App Development. Before we start, let's verify everyone is set up correctly."

[CHECKLIST - Go through each item]

1. Check Node.js:

```
node --version
```

2. Create a new Expo project:

```
npx create-expo-app@latest TestApp  
cd TestApp  
npx expo start
```

3. Verify the app runs:

- QR code should appear in terminal
- Scan with Expo Go or use emulator

[SAY] "If anyone is having issues, raise your hand. Once we're all set up, let's begin."

09:15 - 09:45 | React Native Overview & Architecture

[WRITE ON WHITEBOARD]

Today's Goal: Understand React Native
and build your first app.

[SAY] "React Native is a framework for building native mobile apps using React and JavaScript . Instead of writing separate code for Android and iOS, you write once and deploy everywhere."

[PROJECT/DEMONSTRATE]

```
// React Native
<View>
  <Text>Hello World</Text>
</View>

// Maps to Android
<ViewGroup>
  <TextView>Hello World</TextView>
</ViewGroup>

// Maps to iOS
<UIView>
  <UITextView>Hello World</UITextView>
</UIView>
```

[WRITE ON WHITEBOARD]

How React Native Works

Layer	Description
JavaScript	Your code (React components)
Bridge	Connects JS to native
Native	Platform-specific code (Android/iOS)

[SAY] "React Native brings the React programming paradigm to mobile platforms. It uses the same React concepts - components, props, state - but with native components instead of web components ."

Key Points:

- React Native doesn't use HTML. It uses native components
- Components map directly to platform UI building blocks
- The same code works on both Android and iOS

09:45 - 10:15 | EXERCISE: Setup Check

[SAY] "Let's verify everyone can create and run a React Native app."

[PROJECT/WRITE]

```
# Create a new Expo project
npx create-expo-app@latest HelloWorld

# Navigate to project
cd HelloWorld

# Start the development server
npx expo start
```

[GIVE 15 MINUTES]

[DEBRIEF] "Does everyone have their app running? If you're seeing the QR code, you're ready to go."

[COMMON ISSUES]

Issue	Solution
Expo CLI not found	Use <code>npx expo</code> instead
Port already in use	<code>npx expo start --port 8081</code>
Android emulator not running	Open Android Studio first
iOS simulator not found	Install Xcode

10:15 - 10:45 | Core Components & JSX

[SAY] "React Native uses components instead of HTML. The most common components are View, Text, and Image ."

[WRITE ON WHITEBOARD]

Core Components

Component	Purpose	Web Analog
View	Container for other components	<div>
Text	Display text	<p>
Image	Display images	
ScrollView	Scrollable container	<div> with scroll
TextInput	User text input	<input>

[DEMONSTRATE]

```
import React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Image
        source={{ uri: 'https://reactnative.dev/img/header_logo.svg' }}
        style={styles.logo}
      />
      <Text style={styles.title}>Welcome to React Native!</Text>
      <Text style={styles.subtitle}>Learn once, write anywhere</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  logo: {
    width: 200,
    height: 200,
    marginBottom: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 10,
  },
  subtitle: {
    fontSize: 16,
    color: '#666',
  },
});
```

```
    },  
  });
```

[SAY] "Notice the `StyleSheet.create()` - this is how we style in React Native. It's similar to CSS but with camelCase properties."

10:45 - 11:15 | EXERCISE: Hello World App

[SAY] "Your turn. Create a simple Hello World app."

[PROJECT/WRITE]

```
// App.js  
import React from 'react';  
import { View, Text, StyleSheet } from 'react-native';  
  
export default function App() {  
  return (  
    <View style={styles.container}>  
      <Text style={styles.text}>Hello, World!</Text>  
    </View>  
  );  
}  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    justifyContent: 'center',  
    alignItems: 'center',  
    backgroundColor: '#f5f5f5',  
  },  
  text: {  
    fontSize: 24,  
    color: '#333',  
  },  
});
```

[GIVE 10 MINUTES]

[DEBRIEF] "Let's test your app. Run `npx expo start` and see it on your device or emulator."

[DISCUSSION] "What does `flex: 1` do? It takes all available space. What does `justifyContent: 'center'` do? Centers content vertically."

11:15 - 11:45 | Project Structure & Expo

[SAY] "Let's understand the project structure."

[WRITE ON WHITEBOARD]

Expo Project Structure

```
MyApp/
├── .expo/           # Expo configuration
├── .gitignore
├── app.json         # App configuration
├── assets/         # Images, fonts
├── babel.config.js
├── node_modules/
├── package.json
└── App.js          # Main app component
```

[SAY] "Expo is a Framework that simplifies React Native development . It provides developer tools and a standard library of native modules."

Key Expo Features:

- File-based routing (with Expo Router)
- Standard library of native modules
- Expo Go for instant preview
- EAS for builds and submissions

11:45 - 12:15 | EXERCISE: Explore Project

[SAY] "Explore your project structure."

[PROJECT/WRITE]

Tasks:

1. Open `app.json` - what's in it?
2. Open `package.json` - what dependencies are installed?
3. Open `App.js` - modify the text color
4. See the changes with hot reload

[GIVE 15 MINUTES]

[DEBRIEF] "What did you find in `app.json` ? The app name, icon, and splash screen configuration."

"Hot reload - did your app update automatically when you saved changes? This is one of React Native's best features ."

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Building First App

[SAY] "Let's build a more complete app with multiple components."

[PROJECT/WRITE]

```
// App.js
import React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <View style={styles.header}>
        <Text style={styles.headerText}>My App</Text>
      </View>

      <View style={styles.content}>
        <Image
          source={{ uri: 'https://picsum.photos/200' }}
          style={styles.image}
        />
        <Text style={styles.title}>Hello, React Native!</Text>
        <Text style={styles.description}>
          This is my first React Native app. I'm learning to build mobile apps
          for both Android and iOS with a single codebase.
        </Text>
      </View>

      <View style={styles.footer}>
        <Text style={styles.footerText}>© 2026 My App</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
  },
  header: {
    padding: 20,
    backgroundColor: '#007AFF',
    alignItems: 'center',
  },
  headerText: {
    fontSize: 20,
    fontWeight: 'bold',
    color: 'fff',
  },
});
```

```
    },
    content: {
      flex: 1,
      alignItems: 'center',
      justifyContent: 'center',
      padding: 20,
    },
    image: {
      width: 150,
      height: 150,
      borderRadius: 75,
      marginBottom: 20,
    },
    title: {
      fontSize: 24,
      fontWeight: 'bold',
      marginBottom: 10,
    },
    description: {
      fontSize: 16,
      textAlign: 'center',
      color: '#666',
    },
    footer: {
      padding: 20,
      backgroundColor: '#e0e0e0',
      alignItems: 'center',
    },
    footerText: {
      color: '#666',
    },
  },
});
```

[GIVE 20 MINUTES]

[DEBRIEF] "What does this app have? A header, main content, and footer. This is a common app layout."

14:00 - 15:00 | BUILD: Component Practice

[SAY] "Let's build a reusable component."

[PROJECT/WRITE]

Task: Create a reusable Card component.

```
// src/components/Card.js
import React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';
```

```
export default function Card({ title, description, imageUrl }) {
  return (
    <View style={styles.card}>
      <Image source={{ uri: imageUrl }} style={styles.image} />
      <View style={styles.content}>
        <Text style={styles.title}>{title}</Text>
        <Text style={styles.description}>{description}</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: 'fff',
    borderRadius: 8,
    margin: 10,
    overflow: 'hidden',
    shadowColor: 'black',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 3,
  },
  image: {
    width: '100%',
    height: 200,
  },
  content: {
    padding: 16,
  },
  title: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  description: {
    fontSize: 14,
    color: '#666',
    marginTop: 4,
  },
});
```

[GROUP EXERCISE - 30 minutes]

Teams of 2. Each team:

1. Creates the Card component
2. Renders 3 cards in App.js
3. Customizes the styling

[DEBRIEF] "Great work! You've built your first reusable component. This is the foundation of React Native development."

15:00 - 15:30 | DAY 1 WRAP-UP

[SAY] "Let's recap Day 1."

[WRITE ON WHITEBOARD - Fill in with group]

1. React Native = _____ with _____ components
2. Expo = _____ for React Native development
3. Core Components: _____, _____, _____
4. JSX = _____ inside JavaScript
5. StyleSheet = _____ with camelCase properties
6. Hot Reload = See _____ in real-time

Answers:

1. React, native
2. Framework
3. View, Text, Image
4. Markup
5. CSS
6. Changes

[SAY] "One question: **What surprised you most about React Native today?** "

[CLOSING STATEMENT] "Today we learned the foundations of React Native - what it is, how to set it up, and how to build basic components. Tomorrow we'll dive deeper into core components, styling, and Flexbox layout."

Day 1 Facilitator Cheat Sheet

Mistake	How to Fix
Expo not found	Use <code>npx expo</code> instead of <code>expo</code>
Emulator not working	Check Android Studio installation
Missing imports	<code>import { View, Text } from 'react-native'</code>
Style errors	Use camelCase: <code>backgroundColor</code> not <code>background-color</code>
View vs div	React Native uses View, not div

Day 2: Facilitator Guide

Core Components, Styling & Layout

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 1 Recap	15 min
09:15-09:45	Core Components Deep Dive	30 min
09:45-10:15	EXERCISE: Component Practice	30 min
10:15-10:45	Styling with StyleSheet	30 min
10:45-11:15	EXERCISE: Styling Practice	30 min
11:15-11:45	Flexbox Layout	30 min
11:45-12:15	EXERCISE: Flexbox Practice	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	Touch Handling	60 min
14:00-15:00	BUILD: Complete Profile Screen	60 min
15:00-15:30	Day 2 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 1 Recap

[SAY] "Quick recap from yesterday. **What is React Native?** "

(Answer: Framework for building native mobile apps with React)

"**What's Expo?** "

(Answer: Framework that simplifies React Native development)

"**Name three core components.** "

(Answer: View, Text, Image)

"**What does `StyleSheet.create()` do?** "

(Answer: Creates styles with camelCase properties)

"Excellent. Today we go deeper."

09:15 - 09:45 | Core Components Deep Dive

[SAY] "Let's explore the core components in more detail."

[WRITE ON WHITEBOARD]

View Component

"The View is the most fundamental component - a container that supports layout, styling, touch handling, and accessibility ."

```
<View style={styles.container}>
  <View style={styles.box}>
    <Text>Inside a box</Text>
  </View>
</View>
```

Text Component

"The Text component displays text - it's the only component that can handle text ."

```
<Text style={styles.heading}>Heading</Text>
<Text numberOfLines={2}>This text will be truncated after two lines</Text>
<Text>
  <Text style={styles.bold}>Bold</Text>
  <Text style={styles.italic}> Italic</Text>
</Text>
```

Image Component

"The Image component displays images - local or remote ."

```
// Local image
<Image source={require('./assets/logo.png')} style={styles.logo} />

// Remote image
<Image source={{ uri: 'https://example.com/image.jpg' }} style={styles.logo} />

// With resize mode
<Image
  source={{ uri: 'https://example.com/image.jpg' }}
  style={styles.background}
```

```
      resizeMode="cover"
    />
  />

```

ScrollView Component

"ScrollView provides a scrolling container - useful for content that might exceed the screen."

```
<ScrollView
  horizontal={false}
  showsVerticalScrollIndicator={false}
>
  <View style={styles.item} />
  <View style={styles.item} />
  <View style={styles.item} />
</ScrollView>

```

09:45 - 10:15 | EXERCISE: Component Practice

[SAY] "Build a screen with multiple components."

[PROJECT/WRITE]

```
import React from 'react';
import { View, Text, Image, ScrollView, StyleSheet } from 'react-native';

export default function ComponentsScreen() {
  return (
    <ScrollView style={styles.container}>
      <View style={styles.header}>
        <Image
          source={{ uri: 'https://reactnative.dev/img/header_logo.svg' }}
          style={styles.logo}
        />
        <Text style={styles.title}>React Native</Text>
      </View>

      <View style={styles.section}>
        <Text style={styles.sectionTitle}>Core Components</Text>
        <Text style={styles.sectionText}>
          React Native provides a set of essential components for building mobile apps
        </Text>
      </View>

      <View style={styles.card}>
        <Text style={styles.cardTitle}>Card Example</Text>
        <Text style={styles.cardText}>
          This is a card component created with View, Text, and Image.
        </Text>
      </View>
    </ScrollView>
  );
}

```

```
    </View>
  </ScrollView>
);
}

const styles = StyleSheet.create({
  // Complete with your styles
});
```

[GIVE 20 MINUTES]

10:15 - 10:45 | Styling with StyleSheet

[SAY] "StyleSheet is how we style components in React Native."

[WRITE ON WHITEBOARD]

StyleSheet Syntax

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    padding: 20,
    margin: 10,
  },
});
```

Common Style Properties

Category	Properties
Layout	flex , flexDirection , justifyContent , alignItems
Dimensions	width , height , padding , margin
Text	fontSize , fontWeight , color , textAlign
Background	backgroundColor
Border	borderWidth , borderColor , borderRadius

Platform-Specific Styling

```
import { Platform } from 'react-native';

const styles = StyleSheet.create({
```

```
    container: {  
      padding: Platform.OS === 'ios' ? 20 : 10,  
    },  
  });  
};
```

10:45 - 11:15 | EXERCISE: Styling Practice

[SAY] "Style this profile card."

[PROJECT/WRITE]

```
// Complete the styles  
const styles = StyleSheet.create({  
  card: {  
    // White background  
    // Border radius  
    // Padding  
    // Shadow  
  },  
  image: {  
    // Width 100%  
    // Height 200  
  },  
  content: {  
    // Padding  
  },  
  name: {  
    // Font size 20  
    // Bold  
  },  
  role: {  
    // Gray color  
    // Font size 14  
  },  
});
```

[GIVE 15 MINUTES]

11:15 - 11:45 | Flexbox Layout

[SAY] "Flexbox is the layout system in React Native. Understanding flexbox is crucial for building responsive layouts."

[WRITE ON WHITEBOARD]

Flex Properties

flex - Takes available space :

```
<View style={{ flex: 1 }}>
  <View style={{ flex: 1, backgroundColor: 'red' }} />
  <View style={{ flex: 1, backgroundColor: 'blue' }} />
</View>
```

flexDirection - Direction of children:

- row - horizontal
- column - vertical (default)

justifyContent - Main axis alignment :

- flex-start
- center
- flex-end
- space-between
- space-around

alignItems - Cross axis alignment :

- flex-start
- center
- flex-end
- stretch

11:45 - 12:15 | EXERCISE: Flexbox Practice

[SAY] "Create these layouts using flexbox."

[PROJECT/WRITE]

Layout 1: Header - Content - Footer

```
// Full height, header at top, footer at bottom
<View style={{ flex: 1 }}>
  <View style={styles.header}>Header</View>
  <View style={styles.content}>Content</View>
  <View style={styles.footer}>Footer</View>
</View>
```

Layout 2: Three columns

```
// Three equal columns
<View style={{ flexDirection: 'row', flex: 1 }}>
  <View style={styles.column}>1</View>
  <View style={styles.column}>2</View>
  <View style={styles.column}>3</View>
</View>
```

Layout 3: Centered content

```
// Centered horizontally and vertically
<View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
  <Text>Centered</Text>
</View>
```

[GIVE 15 MINUTES]

Afternoon Session Script

13:00 - 14:00 | Touch Handling

[SAY] "Handling touch events is essential for interactive apps."

[WRITE ON WHITEBOARD]

Button Component

```
<Button
  title="Press Me"
  onPress={() => console.log('Pressed')}
  color="#007AFF"
/>
```

TouchableOpacity

```
<TouchableOpacity
  onPress={() => console.log('Pressed')}
  activeOpacity={0.7}
  style={styles.button}
>
  <Text style={styles.buttonText}>Tap Me</Text>
</TouchableOpacity>
```

Pressable Component

```

<Pressable
  onPress={() => console.log('Pressed')}
  onPressIn={() => console.log('Pressed In')}
  style={({ pressed }) => ({
    backgroundColor: pressed ? '#ddd' : '#007AFF',
    padding: 12,
    borderRadius: 8,
  })}
>
  <Text style={{ color: '#fff' }}>Press Me</Text>
</Pressable>

```

TextInput Component

```

<TextInput
  style={styles.input}
  placeholder="Enter text"
  value={text}
  onChangeText={setText}
  keyboardType="email-address" // or 'numeric', 'phone-pad'
  secureTextEntry={false}
  autoCapitalize="none"
/>

```

14:00 - 15:00 | BUILD: Complete Profile Screen

[SAY] "Build a complete profile screen."

[PROJECT/WRITE]

Requirements:

1. Profile image (use placeholder)
2. Name **and** handle
3. Followers, following, posts counts
4. Bio
5. Follow button
6. Grid of posts (3 columns)

[GROUP EXERCISE - 30 minutes]

Teams of 2-3. Build the complete screen.

[FACILITATOR SOLUTION]

```
import React from 'react';
import { View, Text, Image, ScrollView, TouchableOpacity, StyleSheet } from 'react-native';

const posts = [
  { id: 1, image: 'https://picsum.photos/200/200?random=1' },
  { id: 2, image: 'https://picsum.photos/200/200?random=2' },
  { id: 3, image: 'https://picsum.photos/200/200?random=3' },
  { id: 4, image: 'https://picsum.photos/200/200?random=4' },
  { id: 5, image: 'https://picsum.photos/200/200?random=5' },
  { id: 6, image: 'https://picsum.photos/200/200?random=6' },
];

export default function ProfileScreen() {
  return (
    <ScrollView style={styles.container}>
      <View style={styles.header}>
        <Image
          source={{ uri: 'https://picsum.photos/100/100' }}
          style={styles.avatar}
        />
        <View style={styles.stats}>
          <View style={styles.stat}>
            <Text style={styles.statNumber}>150</Text>
            <Text style={styles.statLabel}>Posts</Text>
          </View>
          <View style={styles.stat}>
            <Text style={styles.statNumber}>1.2K</Text>
            <Text style={styles.statLabel}>Followers</Text>
          </View>
          <View style={styles.stat}>
            <Text style={styles.statNumber}>350</Text>
            <Text style={styles.statLabel}>Following</Text>
          </View>
        </View>
        <View style={styles.profile}>
          <Text style={styles.name}>John Doe</Text>
          <Text style={styles.handle}>@johndoe</Text>
          <Text style={styles.bio}>Mobile developer learning React Native</Text>

          <TouchableOpacity style={styles.followButton}>
            <Text style={styles.followText}>Follow</Text>
          </TouchableOpacity>
        </View>

        <View style={styles.posts}>
          {posts.map(post => (
            <Image
              key={post.id}
              source={{ uri: post.image }}
              style={styles.postImage}
            />
          )
        )}
      </ScrollView>
    );
  }
}
```

```
        />
      )})
    </View>
  </ScrollView>
);
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
  },
  header: {
    flexDirection: 'row',
    padding: 20,
    alignItems: 'center',
  },
  avatar: {
    width: 80,
    height: 80,
    borderRadius: 40,
  },
  stats: {
    flex: 1,
    flexDirection: 'row',
    justifyContent: 'space-around',
  },
  stat: {
    alignItems: 'center',
  },
  statNumber: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  statLabel: {
    color: '#666',
    fontSize: 12,
  },
  profile: {
    paddingHorizontal: 20,
    marginBottom: 20,
  },
  name: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  handle: {
    color: '#666',
    marginTop: 4,
  },
  bio: {
    marginTop: 8,
  },
});
```

```
followButton: {
  backgroundColor: '#007AFF',
  paddingVertical: 10,
  borderRadius: 8,
  alignItems: 'center',
  marginTop: 12,
},
followText: {
  color: '#fff',
  fontWeight: 'bold',
},
posts: {
  flexDirection: 'row',
  flexWrap: 'wrap',
},
postImage: {
  width: '33.33%',
  height: 120,
},
},
```

15:00 - 15:30 | DAY 2 WRAP-UP

[SAY] "Let's recap Day 2."

[WRITE ON WHITEBOARD]

1. Core Components: , , ,
2. StyleSheet: styling with camelCase
3. Flexbox: , , ,
4. Touch Handling: , ,
5. TextInput: User

Answers:

1. View, Text, Image, ScrollView
2. CSS-like
3. flex, flexDirection, justifyContent, alignItems
4. Button, TouchableOpacity, Pressable
5. Input

[SAY] "One question: What's the most important flexbox property? "

(Answer: flex - it takes available space)

[CLOSING STATEMENT] "Today we learned core components, styling, Flexbox layout, and touch handling. Tomorrow we'll build lists and work with forms."

Day 2 Facilitator Cheat Sheet

Mistake	How to Fix
Forgetting flex: 1	"Add flex: 1 to fill available space"
Wrong flexDirection	"Default is column, use row for horizontal"
Missing imports	"Import all components used"
Style property names	"Use camelCase, not kebab-case"
Touchable wrapper	"Wrap text in Text component for buttons"

Day 3: Facilitator Guide

Lists, Forms & User Interactions

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 2 Recap	15 min
09:15-09:45	FlatList & Lists	30 min
09:45-10:15	EXERCISE: FlatList Practice	30 min
10:15-10:45	SectionList & Grouped Data	30 min
10:45-11:15	EXERCISE: SectionList	30 min
11:15-11:45	Form Handling	30 min
11:45-12:15	EXERCISE: Forms & Validation	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	AsyncStorage	60 min
14:00-15:00	BUILD: Notes App	60 min
15:00-15:30	Day 3 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 2 Recap

[SAY] "Quick recap. What is Flexbox used for? "

(Answer: Layout in React Native)

"Name three touch handling components. "

(Answer: Button, TouchableOpacity, Pressable)

"What does flex: 1 do? "

(Answer: Takes all available space)

"What is the TextInput component for? "

(Answer: User text input)

"Excellent. Today we build lists and handle forms."

09:15 - 09:45 | FlatList & Lists

[SAY] "Lists are essential in mobile apps. React Native provides FlatList for efficient list rendering ."

[WRITE ON WHITEBOARD]

ScrollView vs FlatList

Feature	ScrollView	FlatList
Rendering	Renders all items	Renders visible items
Performance	Slow for large lists	Optimized
Memory Usage	High	Low
Use Case	Small lists	Large/dynamic data

[DEMONSTRATE]

```
import { FlatList, View, Text, StyleSheet } from 'react-native';

const DATA = [
  { id: '1', title: 'Item 1' },
  { id: '2', title: 'Item 2' },
  { id: '3', title: 'Item 3' },
];

<FlatList
```

```
data={DATA}
keyExtractor={({item}) => item.id}
renderItem={({ item }) => (
  <View style={styles.item}>
    <Text style={styles.itemText}>{item.title}</Text>
  </View>
)}
contentContainerStyle={styles.list}
/>
```

FlatList Key Props

Prop	Description
data	Array of data
renderItem	Function to render each item
keyExtractor	Unique key for each item
numColumns	Number of columns (grid)
ListHeaderComponent	Header component
ListFooterComponent	Footer component
ListEmptyComponent	Empty state component

09:45 - 10:15 | EXERCISE: FlatList Practice

[SAY] "Build a product list with FlatList."

[PROJECT/WRITE]

```
import React from 'react';
import { FlatList, View, Text, Image, StyleSheet } from 'react-native';

const products = [
  { id: '1', name: 'Product 1', price: 19.99, image: 'https://picsum.photos/200/200?ra' },
  { id: '2', name: 'Product 2', price: 29.99, image: 'https://picsum.photos/200/200?ra' },
  { id: '3', name: 'Product 3', price: 39.99, image: 'https://picsum.photos/200/200?ra' }
];

export default function ProductList() {
  return (
    <FlatList
      data={products}
      keyExtractor={({item}) => item.id}
      renderItem={({ item }) => (
```

```

    <View style={styles.card}>
      <Image source={{ uri: item.image }} style={styles.image} />
      <Text style={styles.name}>{item.name}</Text>
      <Text style={styles.price}>${item.price}</Text>
    </View>
  )}
  numColumns={2}
  columnWrapperStyle={styles.row}
  contentContainerStyle={styles.list}
/>
);
}

```

```

const styles = StyleSheet.create({
  list: {
    padding: 16,
  },
  row: {
    justifyContent: 'space-between',
  },
  card: {
    flex: 1,
    backgroundColor: '#fff',
    margin: 8,
    padding: 12,
    borderRadius: 8,
    alignItems: 'center',
  },
  image: {
    width: 100,
    height: 100,
    borderRadius: 8,
  },
  name: {
    fontSize: 14,
    fontWeight: 'bold',
    marginTop: 8,
  },
  price: {
    fontSize: 16,
    color: '#007AFF',
    marginTop: 4,
  },
},
);

```

[GIVE 20 MINUTES]

10:15 - 10:45 | SectionList & Grouped Data

[SAY] "SectionList is used for grouped data."

[DEMONSTRATE]

```
import { SectionList, Text, View, StyleSheet } from 'react-native';

const SECTIONS = [
  {
    title: 'Fruits',
    data: ['Apple', 'Banana', 'Cherry'],
  },
  {
    title: 'Vegetables',
    data: ['Carrot', 'Broccoli', 'Spinach'],
  },
  {
    title: 'Dairy',
    data: ['Milk', 'Cheese', 'Yogurt'],
  },
];

<SectionList
  sections={SECTIONS}
  keyExtractor={({item, index}) => item + index}
  renderItem={({ item }) => (
    <Text style={styles.item}>{item}</Text>
  )}
  renderSectionHeader={({ section: { title } }) => (
    <Text style={styles.sectionHeader}>{title}</Text>
  )}
/>
```

10:45 - 11:15 | EXERCISE: SectionList

[SAY] "Build a grouped list of items."

[PROJECT/WRITE]

```
// Build a SectionList for a grocery app
// Categories: Fruits, Vegetables, Dairy
// Each category has 3-4 items

// Your code here
```

[GIVE 15 MINUTES]**11:15 - 11:45 | Form Handling**

[SAY] "Forms are essential for user input. Let's build one with validation."

[DEMONSTRATE]

```
import React, { useState } from 'react';
import { View, Text, TextInput, Button, StyleSheet, Alert } from 'react-native';

export default function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};
    if (!email) newErrors.email = 'Email is required';
    if (!email.includes('@')) newErrors.email = 'Invalid email';
    if (!password) newErrors.password = 'Password is required';
    if (password.length < 6) newErrors.password = 'Must be at least 6 characters';

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = () => {
    if (validate()) {
      Alert.alert('Success', 'Form submitted!');
    }
  };

  return (
    <View style={styles.container}>
      <View style={styles.field}>
        <Text style={styles.label}>Email</Text>
        <TextInput
          style={[styles.input, errors.email && styles.inputError]}
          value={email}
          onChangeText={setEmail}
          placeholder="Enter your email"
          keyboardType="email-address"
          autoCapitalize="none"
        />
        {errors.email && <Text style={styles.error}>{errors.email}</Text>}
      </View>

      <View style={styles.field}>
        <Text style={styles.label}>Password</Text>
        <TextInput
          style={[styles.input, errors.password && styles.inputError]}
          value={password}
          onChangeText={setPassword}
          placeholder="Enter your password"
          secureTextEntry
        />
      </View>
    </View>
  );
}
```

```

    />
    {errors.password && <Text style={styles.error}>{errors.password}</Text>}
  </View>

  <Button title="Login" onPress={handleSubmit} />
</View>
);
}

```

11:45 - 12:15 | EXERCISE: Forms & Validation

[SAY] "Build a registration form with validation."

[PROJECT/WRITE]

Requirements:

1. Name field (required)
2. Email field (required, valid email)
3. Password field (required, min 6 chars)
4. Confirm password (must match)
5. Submit button with validation

[GIVE 20 MINUTES]

Afternoon Session Script

13:00 - 14:00 | AsyncStorage

[SAY] "AsyncStorage is React Native's key-value storage for persisting data."

[WRITE ON WHITEBOARD]

Installation

```
npm install @react-native-async-storage/async-storage
```

Basic Operations

```

import AsyncStorage from '@react-native-async-storage/async-storage';

// Save
async function saveData(key, value) {
  try {
    await AsyncStorage.setItem(key, JSON.stringify(value));
  }
}

```

```
    } catch (error) {
      console.error('Error saving:', error);
    }
  }

// Get
async function getData(key) {
  try {
    const value = await AsyncStorage.getItem(key);
    return value ? JSON.parse(value) : null;
  } catch (error) {
    console.error('Error reading:', error);
  }
}

// Remove
async function removeData(key) {
  try {
    await AsyncStorage.removeItem(key);
  } catch (error) {
    console.error('Error removing:', error);
  }
}
```

14:00 - 15:00 | BUILD: Notes App

[SAY] "Build a complete notes app with persistence."

[GROUP EXERCISE - 45 minutes]

Requirements:

1. Add notes with title and content
2. List all notes with FlatList
3. Delete notes (long press)
4. AsyncStorage for persistence
5. Empty state

[FACILITATOR SOLUTION PROVIDED IN CANDIDATE EDITION]

15:00 - 15:30 | DAY 3 WRAP-UP

[SAY] "Let's recap Day 3."

[WRITE ON WHITEBOARD]

- 1. FlatList: _____ list rendering
- 2. SectionList: _____ data with sections
- 3. Forms: _____ **for** user input
- 4. Validation: Check _____ before submission
- 5. AsyncStorage: _____ data on device

Answers:

- 1. Efficient
- 2. Grouped
- 3. TextInput
- 4. Input
- 5. Persistent

[SAY] "One question: **When should you use FlatList vs ScrollView?** "

(Answer: FlatList for large/dynamic data, ScrollView for small lists)

[CLOSING STATEMENT] "Today we learned FlatList, forms, validation, and data persistence with AsyncStorage. Tomorrow we'll learn navigation and state management."

Day 4: Facilitator Guide

Navigation & State Management

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 3 Recap	15 min
09:15-09:45	React Navigation	30 min
09:45-10:15	EXERCISE: Stack Navigator	30 min
10:15-10:45	Tab & Drawer Navigation	30 min
10:45-11:15	EXERCISE: Tab Navigator	30 min
11:15-11:45	State Management: useState/useReducer	30 min
11:45-12:15	EXERCISE: useReducer Practice	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	Context API	60 min

Time	Activity	Duration
14:00-15:00	BUILD: Complete Navigation System	60 min
15:00-15:30	Day 4 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 3 Recap

[SAY] "Quick recap. What is FlatList? "

(Answer: Efficient list rendering component)

"What is SectionList used for? "

(Answer: Grouped data with sections)

"What is AsyncStorage? "

(Answer: Persistent key-value storage)

"How do you validate forms? "

(Answer: Check input before submission)

"Excellent. Today we learn navigation and state management."

09:15 - 09:45 | React Navigation

[SAY] "React Navigation is the standard library for navigation in React Native."

[WRITE ON WHITEBOARD]

Installation

```
npm install @react-navigation/native @react-navigation/native-stack  
npm install react-native-screens react-native-safe-area-context
```

Navigation Container

```
import { NavigationContainer } from '@react-navigation/native';  
import { createNativeStackNavigator } from '@react-navigation/native-stack';  
  
const Stack = createNativeStackNavigator();
```

```
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Navigating Between Screens

```
// HomeScreen.js
export default function HomeScreen({ navigation }) {
  return (
    <View>
      <Button
        title="Go to Details"
        onPress={() => navigation.navigate('Details', {
          id: 1,
          name: 'Item'
        })}
      />
      <Button
        title="Go Back"
        onPress={() => navigation.goBack()}
      />
    </View>
  );
}
```

```
// DetailsScreen.js
export default function DetailsScreen({ route }) {
  const { id, name } = route.params;
  return (
    <View>
      <Text>ID: {id}</Text>
      <Text>Name: {name}</Text>
    </View>
  );
}
```

09:45 - 10:15 | EXERCISE: Stack Navigator

[SAY] "Build a stack navigator with three screens."

[PROJECT/WRITE]

Screens:

1. Home – list of items
2. Details – item details
3. Profile – user profile

Navigation:

- Home → Details (pass item ID)
- Home → Profile
- Back buttons

[GIVE 20 MINUTES]

10:15 - 10:45 | Tab & Drawer Navigation

[SAY] "Tab and drawer navigation provide different navigation patterns."

[WRITE ON WHITEBOARD]

Tab Navigator

```
npm install @react-navigation/bottom-tabs
npm install @expo/vector-icons
```

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';
```

```
const Tab = createBottomTabNavigator();
```

```
<Tab.Navigator
  screenOptions={({ route }) => ({
    tabBarIcon: ({ focused, color, size }) => {
      // Return icon based on route name
    },
    tabBarActiveTintColor: '#007AFF',
    tabBarInactiveTintColor: 'gray',
  })}
>
  <Tab.Screen name="Home" component={HomeScreen} />
  <Tab.Screen name="Settings" component={SettingsScreen} />
</Tab.Navigator>
```

Drawer Navigator

```
npm install @react-navigation/drawer
```

```
import { createDrawerNavigator } from '@react-navigation/drawer';

const Drawer = createDrawerNavigator();

<Drawer.Navigator
  screenOptions={{
    drawerActiveTintColor: '#007AFF',
    drawerStyle: {
      backgroundColor: '#fff',
      width: 240,
    },
  }}
>
  <Drawer.Screen name="Home" component={HomeScreen} />
  <Drawer.Screen name="About" component={AboutScreen} />
</Drawer.Navigator>
```

10:45 - 11:15 | EXERCISE: Tab Navigator

[SAY] "Add bottom tab navigation to your app."

[PROJECT/WRITE]

Tabs:

1. Home – Stack Navigator with screens
2. Search – Search screen
3. Profile – Profile screen

Icons: Use Ionicons

[GIVE 20 MINUTES]

11:15 - 11:45 | State Management: useState/useReducer

[SAY] "State management is crucial for handling data in your app."

[WRITE ON WHITEBOARD]

useState

```
const [state, setState] = useState(initialValue);
```

useReducer

```
const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      return state;
  }
}

const [state, dispatch] = useReducer(reducer, initialState);
```

11:45 - 12:15 | EXERCISE: useReducer Practice

[SAY] "Build a shopping cart with useReducer."

[PROJECT/WRITE]

```
// Cart reducer
// Actions: add, remove, update quantity, clear
// State: items array with quantities

// Your code here
```

[GIVE 20 MINUTES]

Afternoon Session Script

13:00 - 14:00 | Context API

[SAY] "Context API provides global state management."

[WRITE ON WHITEBOARD]

Creating Context

```
// context/AuthContext.js
import React, { createContext, useState, useContext } from 'react';

const AuthContext = createContext();
```

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (userData) => {
    setUser(userData);
  };

  const logout = () => {
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

Using Context

```
// App.js
import { AuthProvider } from '../context/AuthContext';
import AppNavigator from '../navigation/AppNavigator';

export default function App() {
  return (
    <AuthProvider>
      <AppNavigator />
    </AuthProvider>
  );
}

// ProfileScreen.js
import { useAuth } from '../context/AuthContext';

export default function ProfileScreen() {
  const { user, logout } = useAuth();
  // Use user and logout
}
```

14:00 - 15:00 | BUILD: Complete Navigation System

[SAY] "Build a complete navigation system."

[PROJECT/WRITE]

Structure:

1. Auth Stack (Login, Register)
2. Main Tabs (Home, Search, Profile)
3. Stack within Home (Home → Details)
4. Context **for** auth state
5. Protected routes

[GROUP EXERCISE - 45 minutes]**15:00 - 15:30 | DAY 4 WRAP-UP**

[SAY] "Let's recap Day 4."

[WRITE ON WHITEBOARD]

1. React Navigation: ____ library
2. Stack Navigator: ____ navigation
3. Tab Navigator: ____ navigation
4. Drawer Navigator: ____ navigation
5. useState: ____ **state** management
6. useReducer: ____ **state** management
7. Context API: ____ **state** management

Answers:

1. Navigation
2. Screen stacking
3. Bottom tabs
4. Side drawer
5. Local
6. Complex
7. Global

[SAY] "One question: **What's the difference between useState and Context?** "

(Answer: useState is local, Context is global)

[CLOSING STATEMENT] "Today we learned navigation and state management. Tomorrow we'll work with native features, debugging, and publishing."

Day 5: Facilitator Guide

Native Features, Debugging & Publishing

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 4 Recap	15 min
09:15-09:45	Native Features (Camera, Location)	30 min
09:45-10:15	EXERCISE: Camera	30 min
10:15-10:45	Push Notifications	30 min
10:45-11:15	EXERCISE: Notifications	30 min
11:15-11:45	Debugging Tools	30 min
11:45-12:15	EXERCISE: Debugging	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	Testing	60 min
14:00-15:00	Publishing	60 min
15:00-15:30	Final Wrap-Up & Certificates	30 min

Morning Session Script

09:00 - 09:15 | Day 4 Recap

[SAY] "Quick recap. **What is React Navigation?** "

(Answer: Navigation library)

"What is the Context API used for? "

(Answer: Global state management)

"What's the difference between Stack and Tab navigation? "

(Answer: Stack = screen stacking, Tab = bottom tabs)

"What is useReducer? "

(Answer: Complex state management)

"Excellent. Today we learn native features, debugging, and publishing."

09:15 - 09:45 | Native Features

[SAY] "React Native apps can access device features."

[WRITE ON WHITEBOARD]

Camera

```
npx expo install expo-camera
```

```
import { Camera } from 'expo-camera';

// Request permission
const { status } = await Camera.requestCameraPermissionsAsync();

// Take picture
const photo = await cameraRef.current.takePictureAsync();
```

Location

```
npx expo install expo-location
```

```
import * as Location from 'expo-location';

// Request permission
const { status } = await Location.requestForegroundPermissionsAsync();

// Get location
const location = await Location.getCurrentPositionAsync({});
```

Push Notifications

```
npx expo install expo-notifications
```

```
import * as Notifications from 'expo-notifications';

// Request permission
const { status } = await Notifications.requestPermissionsAsync();

// Get token
```

```
const token = await Notifications.getExpoPushTokenAsync();

// Send notification
await Notifications.scheduleNotificationAsync({
  content: { title, body },
  trigger: { seconds: 2 },
});
```

09:45 - 10:15 | EXERCISE: Camera

[SAY] "Build a camera app."

[PROJECT/WRITE]

Features:

1. Camera preview
2. Flip camera button
3. Capture button
4. Display captured image

[GIVE 20 MINUTES]

10:15 - 10:45 | Push Notifications

[SAY] "Push notifications keep users engaged."

[DEMONSTRATE]

```
import React, { useEffect } from 'react';
import { View, Text, Button } from 'react-native';
import * as Notifications from 'expo-notifications';

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: false,
    shouldSetBadge: false,
  }),
});

export default function NotificationDemo() {
  useEffect(() => {
    (async () => {
      const { status } = await Notifications.requestPermissionsAsync();
      if (status === 'granted') {
        const token = await Notifications.getExpoPushTokenAsync();
```

```
        console.log('Push token:', token.data);
    }
  })();
}, []);

const sendNotification = async () => {
  await Notifications.scheduleNotificationAsync({
    content: {
      title: "Hello!",
      body: "This is a test notification",
    },
    trigger: { seconds: 2 },
  });
};

return (
  <View>
    <Text>Notifications Demo</Text>
    <Button title="Send Notification" onPress={sendNotification} />
  </View>
);
}
```

10:45 - 11:15 | EXERCISE: Notifications

[SAY] "Add notifications to your app."

[PROJECT/WRITE]

Features:

1. Request permission
2. Get push token
3. Schedule a notification
4. Handle notification tap

[GIVE 20 MINUTES]

11:15 - 11:45 | Debugging Tools

[SAY] "Debugging is essential for app development."

[WRITE ON WHITEBOARD]

Debugging Tools

Tool	Purpose
React DevTools	Component inspection
Chrome DevTools	JS debugging
React Native Debugger	Combined debugging
Expo Dev Tools	Expo-specific tools

Console Logging

```
console.log('State:', state);  
console.warn('Warning message');  
console.error('Error message');
```

React Native Debugger

```
# Install React Native Debugger  
# Then open and connect
```

11:45 - 12:15 | EXERCISE: Debugging

[SAY] "Practice debugging techniques."

[PROJECT/WRITE]

Tasks:

1. Add console logs to your app
2. Use React DevTools to inspect component
3. Simulate an error and fix it
4. Use the debugger

[GIVE 20 MINUTES]

Afternoon Session Script

13:00 - 14:00 | Testing

[SAY] "Testing ensures your app works correctly."

[WRITE ON WHITEBOARD]

Installing Testing Tools

```
npm install --save-dev jest @testing-library/react-native
```

Unit Tests

```
// __tests__/App.test.js
import React from 'react';
import { render } from '@testing-library/react-native';
import App from '../App';

test('renders welcome text', () => {
  const { getByText } = render(<App />);
  expect(getByText(/Welcome/i)).toBeTruthy();
});
```

Component Tests

```
import { render, fireEvent } from '@testing-library/react-native';
import Button from '../Button';

test('button calls onPress', () => {
  const onPress = jest.fn();
  const { getByText } = render(
    <Button title="Press" onPress={onPress} />
  );

  fireEvent.press(getByText('Press'));
  expect(onPress).toHaveBeenCalled();
});
```

14:00 - 15:00 | Publishing

[SAY] "Publishing makes your app available to users."

[WRITE ON WHITEBOARD]

Expo EAS Build

```
# Install EAS CLI
npm install -g eas-cli
```

```
# Login
eas login
```

```
# Configure
eas build:configure

# Build Android
eas build --platform android

# Build iOS
eas build --platform ios

# Submit
eas submit --platform android
eas submit --platform ios
```

Android Publishing Steps

1. Generate keystore
2. Configure signing
3. Build AAB
4. Upload to Google Play Console

iOS Publishing Steps

1. Get Apple Developer account
2. Create certificates
3. Build IPA
4. Upload to App Store Connect

15:00 - 15:30 | FINAL WRAP-UP

[SAY] "Congratulations on completing the React Native course!"

[WRITE ON WHITEBOARD - Full Summary]

REACT NATIVE FOUNDATIONS	
DAY 1: FOUNDATIONS	
– React Native, Expo, Architecture	
– Core Components: View, Text, Image	
– JSX, StyleSheet, Project Structure	
DAY 2: COMPONENTS & LAYOUT	
– Core Components deep dive	
– Styling with StyleSheet	

- Flexbox layout
- Touch handling: Button, TouchableOpacity

DAY 3: LISTS & FORMS

- FlatList for efficient lists
- SectionList for grouped data
- Form handling and validation
- AsyncStorage for persistence

DAY 4: NAVIGATION & STATE

- React Navigation
- Stack, Tab, Drawer navigation
- useState, useReducer, Context API

DAY 5: NATIVE FEATURES & PUBLISHING

- Camera, Location, Notifications
- Debugging tools
- Testing with Jest
- Publishing with EAS

KEY INSIGHT: React Native = Learn once, write anywhere
Rule: Build with React, run natively.

[SAY] "Let's do a final Q&A."

Common Questions:

Q: "When should I use Expo vs React Native CLI?"

A: "Expo for starting new apps, React Native CLI for advanced native code"

Q: "How do I add native libraries?"

A: "Expo has modules, or use expo-dev-client"

Q: "How much does it cost to publish?"

A: "Google Play: \$25 one-time. Apple: \$99/year."

[DISTRIBUTE] Give each participant a completion certificate.

[FINAL CLOSING STATEMENT] "React Native is a powerful framework for building cross-platform mobile apps. You've learned the foundations - components, styling, navigation, and native features. Now go build something amazing!"

"Remember: **React Native = Learn once, write anywhere.** "

Day 5 Facilitator Cheat Sheet

Mistake	How to Fix
Camera permission denied	Add to AndroidManifest/Info.plist
Location not working	Request permission first
Notifications not firing	Check permissions and trigger
Debugging not connecting	Check port and network
Build failing	Check configuration and dependencies

Appendix: Answer Keys

Day 1 Answers

Exercise 1.1: Hello World

✅ App created with `npx create-expo-app` ✅ App runs on device/emulator ✅ Styling works

Exercise 1.2: Component Practice

✅ Image displayed ✅ Text displayed with styling ✅ Proper layout

Exercise 1.3: Project Structure

✅ `src` directory created ✅ Components organized

Day 2 Answers

Exercise 2.1: Profile Card

```
const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    borderRadius: 12,
    padding: 16,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 3,
  },
  // Rest of styles
});
```

Day 3 Answers

Exercise 3.1: FlatList

✅ Data displayed ✅ numColumns works ✅ Pull-to-refresh implemented

Exercise 3.3: Notes App

✅ AsyncStorage works ✅ CRUD operations complete ✅ UI functional

Day 4 Answers

Exercise 4.1: Stack Navigation

✅ All screens navigable ✅ Params passed correctly ✅ Back navigation works

Exercise 4.4: Combined Navigation

✅ Auth flow works ✅ Main tabs render ✅ Stack within tabs

Day 5 Answers

Exercise 5.1: Camera

✅ Permission requested ✅ Camera preview works ✅ Photo captured

Exercise 5.4: Build

✅ APK generated ✅ App installs ✅ App runs on device

Appendix: Environment Setup Guide

Complete Setup Commands

```
# Install Node.js (download from nodejs.org)

# Install Expo CLI
npm install -g expo-cli

# Create a new project
npx create-expo-app@latest MyApp

# Navigate and start
cd MyApp
npx expo start

# Install common dependencies
npm install @react-navigation/native @react-navigation/native-stack
npm install react-native-screens react-native-safe-area-context
```

```
npm install @react-native-async-storage/async-storage
npm install @expo/vector-icons
```

Android Studio Setup (for Android Emulator)

- 1. Download Android Studio from developer.android.com
- 2. Install with default settings
- 3. Open Android Studio
- 4. Create virtual device (AVD Manager)
- 5. Start the emulator

Xcode Setup (for iOS Simulator - Mac only)

- 1. Download Xcode from App Store
- 2. Open Xcode and accept license
- 3. Install additional components
- 4. Open preferences and add a simulator

VS Code Extensions

- React Native Tools
- ESLint
- Prettier
- EditorConfig

Troubleshooting Common Issues

Issue	Solution
Port already in use	<code>npx expo start --port 8081</code>
Metro bundler error	Delete <code>node_modules</code> and reinstall
Android emulator not working	Check Android Studio installation
iOS simulator not working	Check Xcode installation
Expo Go not connecting	Check same network